INFORME DE INCIDENCIA PHP2SHELL

Diagnóstico, mitigación y actualización de PHP en Kali Linux

Autor: Adrián Molina

Fecha de actualización documentada: 23/10/2026

Estructura del informe

1. Datos del sistema: nombre de la máquina (hostname), distribución (lsb_release -d), versión de PHP y repositorio de origen.
2. Diagnóstico inicial: ¿era vulnerable el sistema? Justifícalo con la evidencia del changelog.
3. Acciones realizadas: lista de lo que has hecho, en orden y con fecha y hora.
4. Verificación: prueba de que el sistema está corregido y de que Apache se reinició después de la actualización.
5. Riesgo residual: ¿queda algo pendiente? Por ejemplo, la mitigación sigue activa, o el parche aún no estaba disponible.
6. Evidencias: las 6 evidencias numeradas, cada una con una frase que explique qué demuestra.

Preguntas de reflexión

Responde con tus propias palabras, en 3 o 4 líneas cada una.

1. Tu compañero dice: «Mi php -v sigue mostrando 8.3.6, así que no estoy parcheado». ¿Tiene razón? ¿Cómo lo comprobarías?
2. ¿Por qué un servidor con el paquete ya actualizado puede seguir siendo vulnerable? ¿Qué comando lo evita?
3. ¿Qué diferencia hay entre una mitigación y una corrección? Pon un ejemplo de esta práctica para cada una.
4. Una empresa tiene un servidor con PHP 7.4 que «funciona perfectamente y nadie quiere tocar». ¿Qué riesgo corre con php2shell y qué le recomendarías?
5. Los detalles de estas vulnerabilidades se filtraron antes de que hubiera parche. ¿Qué es una divulgación coordinada y por qué es importante respetarla?
6. Si administraras 50 servidores en lugar de uno, ¿harías este proceso a mano en cada uno? Investiga y nombra una herramienta que permita automatizarlo.

Rúbrica de evaluación

Criterio	Peso	Excelente	Suficiente	Insuficiente
Diagnóstico (versión, origen, changelog)	20 %	Identifica versión y repositorio, y justifica el estado con el changelog	Identifica la versión, pero la justificación es incompleta	Se basa solo en el número de versión
Mitigación	15 %	Copia de seguridad, cambio correcto y verificado en info.php	Cambio correcto sin verificación	No aplicada o incorrecta
Actualización	20 %	Revisa antes de actualizar y conserva el php.ini	Actualiza correctamente sin revisión previa	No actualiza o rompe el servicio
Reinicio y verificación	20 %	Demuestra con fechas que el reinicio fue posterior	Reinicia, pero no lo demuestra	No reinicia
Limpieza	5 %	Borra info.php y comprueba el 404	Lo borra sin comprobar	No lo borra
Informe	10 %	Completo, ordenado y con evidencias explicadas	Completo pero con evidencias sin explicar	Faltan apartados
Preguntas de reflexión	10 %	Respuestas razonadas y propias	Respuestas correctas pero escuetas	Sin responder o copiadas

1. Datos del sistema

Campo	**Dato documentado**
Nombre de máquina (hostname)	kaliMolina
Distribución (lsb_release -d)	Kali GNU/Linux Rolling
PHP inicial	PHP 8.4.24
PHP tras la actualización	PHP 8.4.27
Servidor web	Apache 2 (servicio apache2); el estado descrito es active (running).
Repositorio de origen	http://http.kali.org/kali

2. Diagnóstico inicial

El diagnóstico inicial se considera vulnerable con la evidencia descrita en el trabajo: la búsqueda de los identificadores CVE-2026-109618 y CVE-2026-109619 en el changelog del paquete instalado no devolvió resultados. Ausencia de ambos CVE ya que el paquete instalado todavía no tenía registrada la actualización. La ausencia de coincidencias no demuestra la vulnerabilidad; debe contrastarse con el changelog completo y con la versión candidata del repositorio.

La consulta del changelog candidato no encontró los CVE inicialmente. Después, el historial de APT muestra una actualización de los paquetes PHP 8.4 desde 8.4.24 a 8.4.27 el 23/10/2026 a las 09:45:19. Una búsqueda posterior del changelog identificó la corrección. Para una auditoría sólida, conviene adjuntar la salida exacta que contiene los CVE y no solo la explicación escrita.

Conclusión inicial: existía una exposición potencial y no se había acreditado el parche mediante el changelog inicial; por ello se aplicó una mitigación temporal y después se actualizaron los paquetes PHP.

3. Acciones realizadas

1. Preparación del entorno. Se instaló Apache y PHP en Kali Linux porque no estaban presentes inicialmente. Se creó la página temporal /var/www/html/info.php con phpinfo() para comprobar la versión que utiliza Apache.

2. Identificación. Se consultó php -v, se identificó la rama de PHP y se revisaron los paquetes instalados con dpkg y apt policy.

```
PHP 8.4.27 (cli) (built: Oct 21 2026 04:47:05) (NTS)
  Copyright (c) The PHP Group
  Built by Debian
  Zend Engine v4.4.27, Copyright (c) Zend Technologies
     with Zend OPcache v8.4.27, Copyright (c), by Zend Technologies
```

3. Diagnóstico por changelog. Se buscó CVE-2026-109618 y CVE-2026-109619 en el changelog instalado y en el candidato. Las búsquedas iniciales descritas no devolvieron coincidencias.

4. Copia de seguridad. Se hizo una copia de /etc/php/$PHPV/apache2/php.ini a php.ini.bak antes de editar la configuración.

5. Mitigación. Se cambió session.upload_progress.enabled a 0 en el php.ini de Apache. Se indica que no existía una configuración PHP-FPM en esa máquina.

6. Comprobación y primer reinicio. Se usó grep -n para revisar la directiva y se reinició Apache con systemctl restart apache2. La página de diagnóstico mostraba el valor Off.

7. Revisión de actualizaciones. Se ejecutó apt update y se revisaron los paquetes PHP que se podían actualizar. [Una actualización general se ejecutó antes de completar esta comprobación por ello no aparecía ninguna respuesta relacionada con una actualización de php.]

8. Actualización de PHP. El historial de APT registra el 23/10/2026 a las 09:45:19 la actualización de paquetes PHP 8.4 de 8.4.24 a 8.4.27, incluyendo CLI, common y el módulo de Apache, además de otros componentes PHP.

9. Reinicio y estado. Reinicio de Apache y una consulta de systemctl status apache2, cuyo resultado se describe como active (running).

10. Limpieza. Se eliminó info.php y se comprobó la respuesta de un 404 en el navegador.

systemctl show apache2 -p ActiveEnterTimestamp; este comando demostrará el último reinicio, lo que respondió con un ActiveEnterTimestamp=Fri 2026-10-23 16:56:39 CEST

4. Verificación

La actualización registrada por APT acredita que los paquetes PHP 8.4 pasaron de 8.4.24 a 8.4.27. Se llevó a cabo una búsqueda posterior en el changelog que encontró la referencia a la corrección de la vulnerabilidad. Esta es la evidencia relevante para comprobar el parche; el número de versión, aislado, no sustituye la comprobación del changelog del paquete concreto.

Respecto a Apache, el trabajo registra el comando sudo systemctl restart apache2 y describe el estado posterior como active (running).

Verificaciones que conviene adjuntar para completar la prueba:

• Salida de zcat /usr/share/doc/php8.4-common/changelog.Debian.gz | grep -E 'CVE-2026-10961[89]'

```
  + (CVE-2026-109618, CVE-2026-109619): Avoid state corruption via
```

• Salida de systemctl status apache2 --no-pager con active (running).

```
apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
     Active: active (running) since Fri 2026-10-23 16:56:39 CEST; 24min ago
 Invocation: a191ef11912640a599ca687953612fac
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 630 (apache2)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 6 (limit: 2278)
     Memory: 26.8M (peak: 27.1M)
        CPU: 416ms
     CGroup: /system.slice/apache2.service
             ├─630 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─744 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─745 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─746 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─747 /usr/sbin/apache2 -k start -DFOREGROUND
             └─748 /usr/sbin/apache2 -k start -DFOREGROUND
```

• Registro de APT con la fecha/hora de la actualización y salida completa de systemctl show apache2 -p ActiveEnterTimestamp que demuestre que el arranque fue posterior.

```
ActiveEnterTimestamp=Fri 2026-10-23 16:56:39 CEST
```

• Comprobación web posterior a la actualización de que PHP responde y que session.upload_progress.enabled sigue en Off.

• Captura o salida de la petición a /info.php muestra un 404 después de eliminar el fichero.

Ambas demostraciones se encuentran en el documento de github.

session.upload_progress.enabled sigue en Off.
/info.php con 404

5. Riesgo residual

La mitigación session.upload_progress.enabled = 0 reduce una vía de ataque relacionada con la gestión del progreso de subida, pero no corrige por sí misma la vulnerabilidad CVE-2026-109618. Por ello, mantener la mitigación no sustituye la instalación y verificación del paquete corregido. Se ha actualizado de PHP 8.4.24 a 8.4.27 y se confirma que el changelog posterior identifica la corrección.

6. Evidencias

Evidencia 1. Página de diagnóstico PHP
Demuestra qué versión de PHP sirve Apache



Evidencia 2. Versión y origen del paquete
Se consultaron php -v, dpkg y apt policy. Permite identificar versión instalada, candidata y repositorio, aunque la salida completa de origen no está reproducida en el texto. Pero el comando de dpkg no funciona

¡Evidencia 3. Changelog inicial y candidato
Las búsquedas iniciales de ambos CVE no respondieron con coincidencias. Esto da a entender que hay una exposición potencial y la necesidad de mitigar y actualizar.

Evidencia 4. Mitigación y reinicio inicial
Se hizo una copia de seguridad de php.ini, el cambio de session.upload_progress.enabled = 0, la comprobación con grep -n y el reinicio de Apache.

Evidencia 5. Actualización y changelog posterior
El historial de APT registra el 23/10/2026 a las 09:45:19 la actualización de PHP 8.4.24 a 8.4.27. El changelog posterior identifica la corrección.

Evidencia 6. Estado de Apache, cronología y limpieza
El estado de Apache se describe como active (running). Para cerrar esta evidencia debe adjuntarse la marca temporal exacta del último arranque, el registro de actualización y la comprobación 404 tras borrar info.php. Como se dijo anteriormente Apache está en estado de active (running), y como se ve el repositorio de github, la última actualización se muestra con el comando de systemctl show apache2 -p ActiveEnterTimestamp y la limpieza también se demuestra con la captura de la imagen del error 404 en info.php en localhost.

Respuestas a las preguntas de reflexión (apartado 9)

1. Si `php -v` sigue mostrando 8.3.6, ¿significa que no está parcheado?

No necesariamente. Algunas distribuciones mantienen la versión base, y aplican la actualización de seguridad sin cambiar el número principal que muestra php, se debería de comprobar revisando el origen y la versión del paquete usando el comando de apt policy y buscando los CVE en el changelog.

2. ¿Por qué un paquete actualizado puede seguir siendo vulnerable? ¿Qué comando lo evita?

Eso puede pasar porque el proceso o servicio puede seguir usando la memoria del código antiguo aunque se haya actualizado. Para comprobarlo, comprobar la actualización, habría que reiniciar el servicio de php con el comando de “sudo systemctl restart apache2” y después habría que comprobar su estado con un status y su último arranque.

3. Diferencia entre mitigación y corrección; un ejemplo de cada una.

Una mitigación reduce la posibilidad de que se pueda explotar una vulnerabilidad, pero no elimina el problema a diferencia de una corrección, Un ejemplo de mitigación sería implementar inicio de sesión de doble contraseña o algo que aumente la seguridad hasta cierto punto, aumenta la seguridad pero no elimina la posibilidad de que alguien encuentre una forma de entrar, Una correción se trataría de una actualización que elimina de raíz el problema, en el caso del ejemplo anterior podría ser una actualización que obliga a los empleados a usar una clave encriptada.

4. Riesgo para una empresa que mantiene PHP 7.4 porque funciona bien.

Que funcione no significa que sea segura, ya que puede haber una vulnerabilidad, puede que hayan ocurrido ya ataques pero no se han detectado, eso me recuerda al informe de Abraham Wald acerca de dónde colocar blindaje en los aviones después de la segunda guerra mundial, solo se concentraba en los que llegaban de vuelta, analizando los daños tomaba en cuenta donde podían recibir disparos los aviones y seguir volando, no sabría explicar la correlación pero tienen algo en común, ya que si algo sigue funcionando, no significa que siga funcionando porque sea perfecto, puede parecer perfecto porque no conoces sus imperfecciones.

5. ¿Qué es la divulgación coordinada y por qué importa?

Es el proceso en el que cuando se descubre una vulnerabilidad, se avisa de forma privada al proveedor, para evitar que atacantes se aprovechen de esta vulnerabilidad mientras se lleva a cabo el parche, es importante por eso. Cuando se ha corregido la vulnerabilidad simplemente se publica el parche, todo el mundo lo actualiza y la mayoría de la gente ni siquiera se entera de la vulnerabilidad hasta que se declara, algo que no pasó con la vulnerabilidad de php, ya que al parecer alguien lo filtró, provocando de php se adelantara a la hora de crear el parche.

6. ¿Automatizarías el proceso en 50 servidores? Nombra una herramienta.

No, aumentaría el tiempo y el riesgo de errores. Usaría Ansible (herramienta de código abierto de automatización de tecnologías de la información) para ejecutar tareas automatizadas en varios servidores, como comprobar versiones, actualizar paquetes, desplegar configuraciones y reiniciar servicios. Antes de aplicarlo a todos, probaría el playbook en un entorno de ensayo y verificaría el resultado en cada máquina. Playbook es un manual operativo o guía práctica que recopila instrucciones paso a paso, estrategias, reglas y mejores prácticas para resolver un problema específico o actuar ante una situación determinada.